Tecnologia Analise e Desenv. de Sistemas

Tratar de Valores Ausentes

Prof. Dr. Ricardo Luis Balieiro

Análise Exploratória de Dados - EDA

As principais etapas da EDA são:

01. Compreender o problema
02. Explorar o conjunto de dados
03. Tratar de valores ausentes
04. Remover registros duplicados
05. Detectar valores discrepantes - Outliers
06. Estudar as Distribuições de Características
07. Explorar as relações entre as variáveis
08. Preparar funcionalidades
09. Verificar o equilíbrio das classes
10. Dividir o conjunto de dados

Valores ausentes

Os valores ausentes podem ocorrer por diversos motivos:

- Formulários incompletos.
- Coleta de dados falhou.
- Erros do sistema.
- Informações indisponíveis.

Primeiros passos

Lidar com valores ausentes geralmente envolve:

- Entender o contexto empresarial.
- Medir a quantidade de dados que faltam.
- Decidir se deve remover, substituir ou preservar.
- Raramente existe uma única solução universal.

Por que os dados estão faltando?

- Porque a solução correta depende do contexto.
- Às vezes, valores ausentes são inofensivos.
- Outras vezes, podem afetar seriamente os modelos de análise ou de aprendizado de máquina.

IMPORTANTE:

- Tratar valores ausentes não se trata de preencher lacunas cegamente, mas sim de tomar decisões informadas sobre dados incompletos.

Tratamento inadequado de dados faltantes

O tratamento inadequado de dados faltantes pode levar a:

- Resultados tendenciosos.
- Modelos imprecisos.
- Análise enganosa.

- Trate os valores ausentes como informação - e não apenas como lacunas.

Tratar com valores ausentes

Como lidar com valores ausentes:

- Identificar quais são os valores ausentes.
- Entender o motivo da ausência dos dados.
- Decidir por remover, substituir ou imputar.

Métodos Comuns de Imputação

- Média, Mediana, Moda
- Preenchimento por modelos
  - Usa previsões de algoritmos sobre outras colunas.

Verificando valores ausentes

Antes de transformar os dados, identifique o tamanho do problema:

- **isna( )** → Retorna um DataFrame de valores booleanos, onde **True** indica dados **ausentes** (NaN).
- **notnull( )** → (inverso de **isna( )**) Retorna um DataFrame com valores booleanos, onde **True** indica dados **presentes** (não ausentes).
- **isnull( )** → Igual ao isna( ). Isso foi feito para manter a familiaridade para quem já utilizava R.

isna( )

Dados Originais

Seleção linhas:

	cpf_cliente	valor_compra	feedback_texto
0	111.222.333-00	150.0	Ótimo
1	222.333.444-11	200.0	NaN
2	333.444.555-22	120.0	NaN
3	444.555.666-33	310.0	NaN
4	555.666.777-44	250.0	NaN
5	666.777.888-55	90.0	NaN
6	777.888.999-66	400.0	NaN
7	888.999.000-77	180.0	NaN
8	999.000.111-88	210.0	Excelente
9	NaN	175.0	Bom

Dados Ausentes

Dados ausentes:

	cpf_cliente	valor_compra	feedback_texto
0	False	False	False
1	False	False	True
2	False	False	True
3	False	False	True
4	False	False	True
5	False	False	True
6	False	False	True
7	False	False	True
8	False	False	False
9	True	False	False

06 - Relacao vendas.csv

notnull( )

Dados Originais

Seleção linhas:

	cpf_cliente	valor_compra	feedback_texto
0	111.222.333-00	150.0	Ótimo
1	222.333.444-11	200.0	NaN
2	333.444.555-22	120.0	NaN
3	444.555.666-33	310.0	NaN
4	555.666.777-44	250.0	NaN
5	666.777.888-55	90.0	NaN
6	777.888.999-66	400.0	NaN
7	888.999.000-77	180.0	NaN
8	999.000.111-88	210.0	Excelente
9	NaN	175.0	Bom

Dados Presentes

Dados presentes:

	cpf_cliente	valor_compra	feedback_texto
0	True	True	True
1	True	True	False
2	True	True	False
3	True	True	False
4	True	True	False
5	True	True	False
6	True	True	False
7	True	True	False
8	True	True	True
9	False	True	True

06 - Relacao vendas.csv

isna( )

Seleção linhas:

	PassengerId	Survived	Pclass	Name	...	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	...	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	...	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	...	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	...	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	...	373450	8.0500	NaN	S

Dados ausentes:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	False	False	False	False	False	False	False	False	False	False	True	False
1	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	True	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	True	False

02 - Titanic-train.csv

Código

```python
import pandas as pd

#============================================================
#                    L E I T U R A
#============================================================
# Carregando o Iris Dataset baixado do Kaggle
df = pd.read_csv("Dados/06 - Relacao vendas.csv")
# df = pd.read_csv("Dados/02 - Titanic-train.csv")


#============================================================
#                  INSPEÇÃO GERAL
#============================================================
# print("\nTamanho do Dataset (Linhas, Colunas):")
# print(df.shape)

# print("\n\nTipo das Variáveis\nDados:")
# print(df.dtypes)

# print("\nSeleção linhas:")
# print(df.head(10))

# print("\nResumo Estatístico (Média, Mín, Máx...):")
# print(df.describe())
```

Código

```
#=========================================================
#              INSPEÇÃO DE VALORES AUSENTES
#=========================================================
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Exemplo 01
# Exibe todos os dados de todas as linhas e colunas
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# # True indica dados AUSENTES
# print("\n\nDados ausentes:")
# print(df.isna())

# # True indica dados PRESENTES
# print("\n\nDados presentes:")
# print(df.notnull())
```

Código

```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Exemplo 02
# Exibe apenas por Linha ou apenas por Coluna
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# # Identifica quais COLUNAS TÊM pelo menos um NaN
# # axis=0 ==> Coluna por Coluna
# coluna_nan = df.isna().any(axis=0)
# print(f"\n Colunas:\n {coluna_nan}")

# # Identifica quais LINHAS TÊM pelo menos um NaN
# # axis=1 ==> Linha por Linha
# linhas_nan = df.isna().any(axis=1)
# print(f"\n Linhas:\n {linhas_nan}")
```

Código

```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Exemplo 03
# Exibe Qtdes ou Porcentagem por coluna
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
total_nulos_col = df.isna().sum()
# print(f"\n\nTotal de nulos por coluna:\n {total_nulos_col}")

percentual_nulos_col = (df.isna().sum() / len(df)) * 100
# print(f"\n\nPercentual de nulos por coluna:\n {percentual_nulos_col}")

# Junta lado a lado (criando um DataFrame com 2 colunas)
df_conj_nulos = pd.concat([total_nulos_col, percentual_nulos_col], axis=1, keys=['qtde', 'porcentagem'])
print(f"\n\nConjunto de nulos:\n {df_conj_nulos}")
```

Abordagens: Remover vs. Imputar

Após o diagnostico dos dados faltantes, deve-se escolher entre dois caminhos estratégicos fundamentais:

- **Remoção (Descarte)**
- **Imputação (Preenchimento)**

Remoção (Descarte)

Remoção (Descarte)

- Deletar registros ou colunas com lacunas.
- Solução mais rápida e simples.
- Resulta em perda de informação.
- Cuidado com redução da amostra e viés.

Quando remover linhas?

Remoção de Linhas (df.dropna(subset=['coluna'])):

- **Regra A ➔ Obrigatório** para identificadores únicos (como id_cliente, cpf), onde não se pode inventar valores.
- **Regra B ➔ Indicado** quando a perda representa menos de **5 %** da base total.

	cpf_cliente	valor_compra	feedback_texto
0	111.222.333-00	150.0	Ótimo
1	222.333.444-11	200.0	NaN
2	333.444.555-22	120.0	NaN
3	444.555.666-33	310.0	NaN
4	555.666.777-44	250.0	NaN
5	666.777.888-55	90.0	NaN
6	777.888.999-66	400.0	NaN
7	888.999.000-77	180.0	NaN
8	999.000.111-88	210.0	Excelente
9	NaN	175.0	Bom

Conjunto de nulos:

	qtde	porcentagem
cpf_cliente	1	10.0
valor_compra	0	0.0
feedback_texto	7	70.0

Quando remover colunas?

Remoção de Colunas (df.drop(columns=['coluna'])):

- **Regra C** ➔ Indicado quando a coluna possui mais de **50 %** a **60 %** de dados faltantes e não há como recuperar a informação original de forma confiável.

	cpf_cliente	valor_compra	feedback_texto
0	111.222.333-00	150.0	Ótimo
1	222.333.444-11	200.0	NaN
2	333.444.555-22	120.0	NaN
3	444.555.666-33	310.0	NaN
4	555.666.777-44	250.0	NaN
5	666.777.888-55	90.0	NaN
6	777.888.999-66	400.0	NaN
7	888.999.000-77	180.0	NaN
8	999.000.111-88	210.0	Excelente
9	NaN	175.0	Bom

Conjunto de nulos:

	qtde	porcentagem
cpf_cliente	1	10.0
valor_compra	0	0.0
feedback_texto	7	70.0

Código

```
#============================================================
#         REMOÇÃO LIN x COL ==> VALORES AUSENTES
#============================================================
# REGRA A: Remover a LINHA onde o identificador único é nulo
# Informação essencial para identificar o comprador -> Não se deve inventar um CPF
df_removido = df.dropna(subset=["cpf_cliente"])
print(f"\n\nDF Removido Linha:\n {df_removido}")

# REGRA B: Remover a COLUNA que ultrapassa o limite de 50% de dados ausentes
# Informação muito incompleta para imputar
df_removido = df_removido.drop(columns=["feedback_texto"])
print(f"\n\nDF Removido Coluna:\n {df_removido}")
```

Atividade Fixação

- **Exercício 1: Diagnóstico e Remoção Crítica**
- Exercício 2: Escolha da Métrica Estatística
- Exercício 3: Tratamento de Séries Temporais

Imputação (Preenchimento)

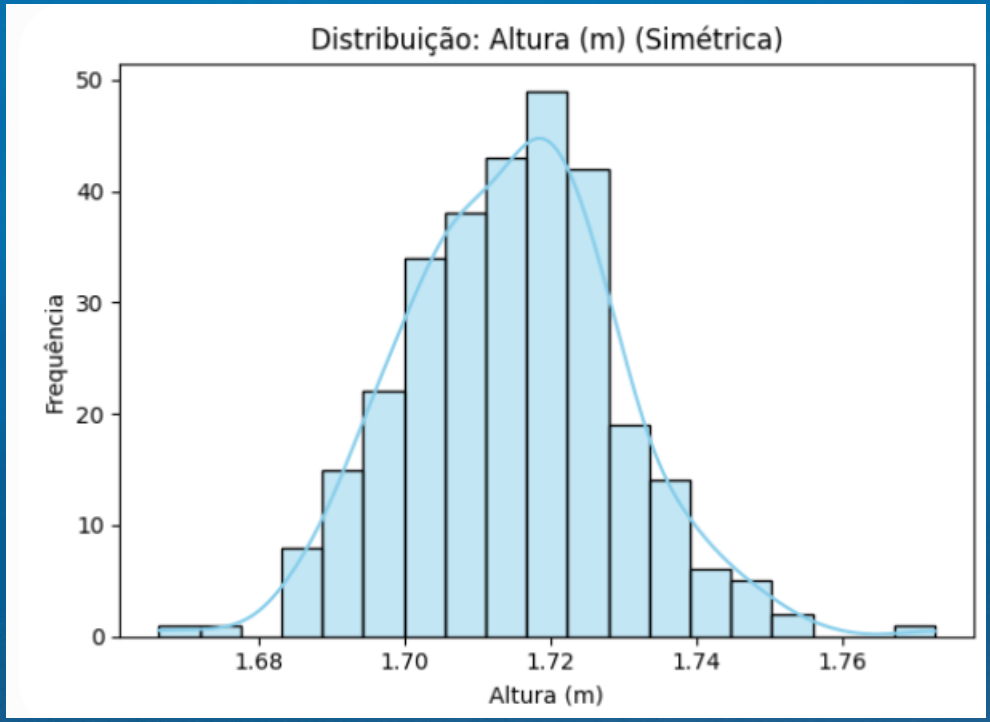
Imputação (Preenchimento)

- Esta abordagem preserva o tamanho da base.
- Estimar valores baseados no restante dos dados.
- Usa estatística descritiva ou lógica temporal.
- Exige cuidado na escolha do método.
  - Objetivo: não distorcer a realidade dos dados.

Imputar dados numéricos

Valores numéricos:

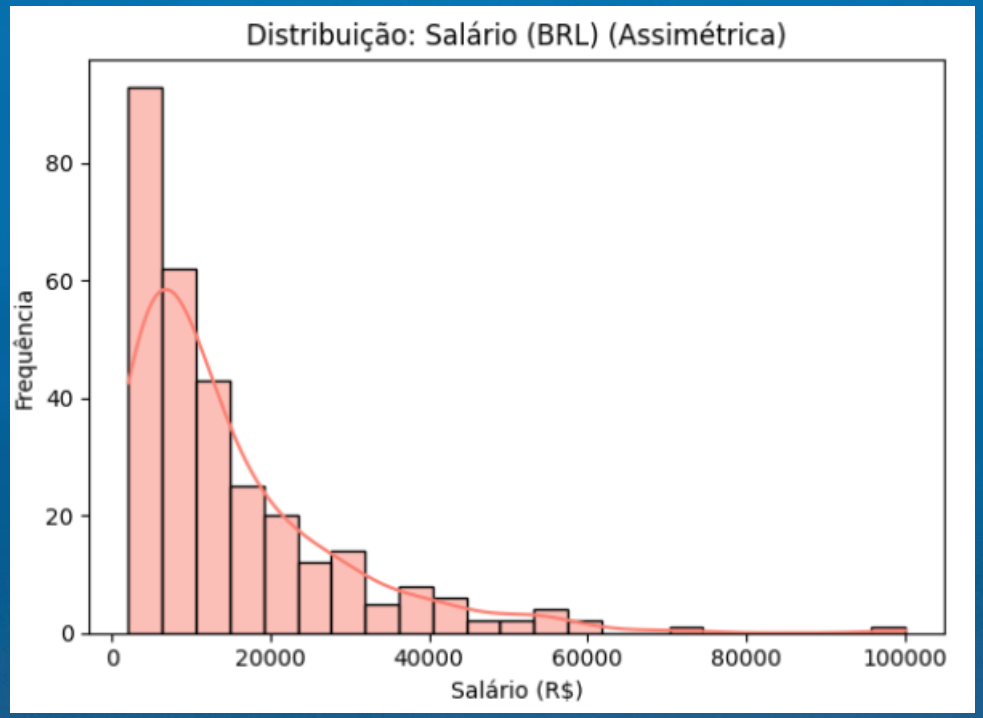
- **Média (.mean())** ➔ Para variáveis numéricas contínuas com distribuição normal/simétrica (sem valores extremos/outliers).



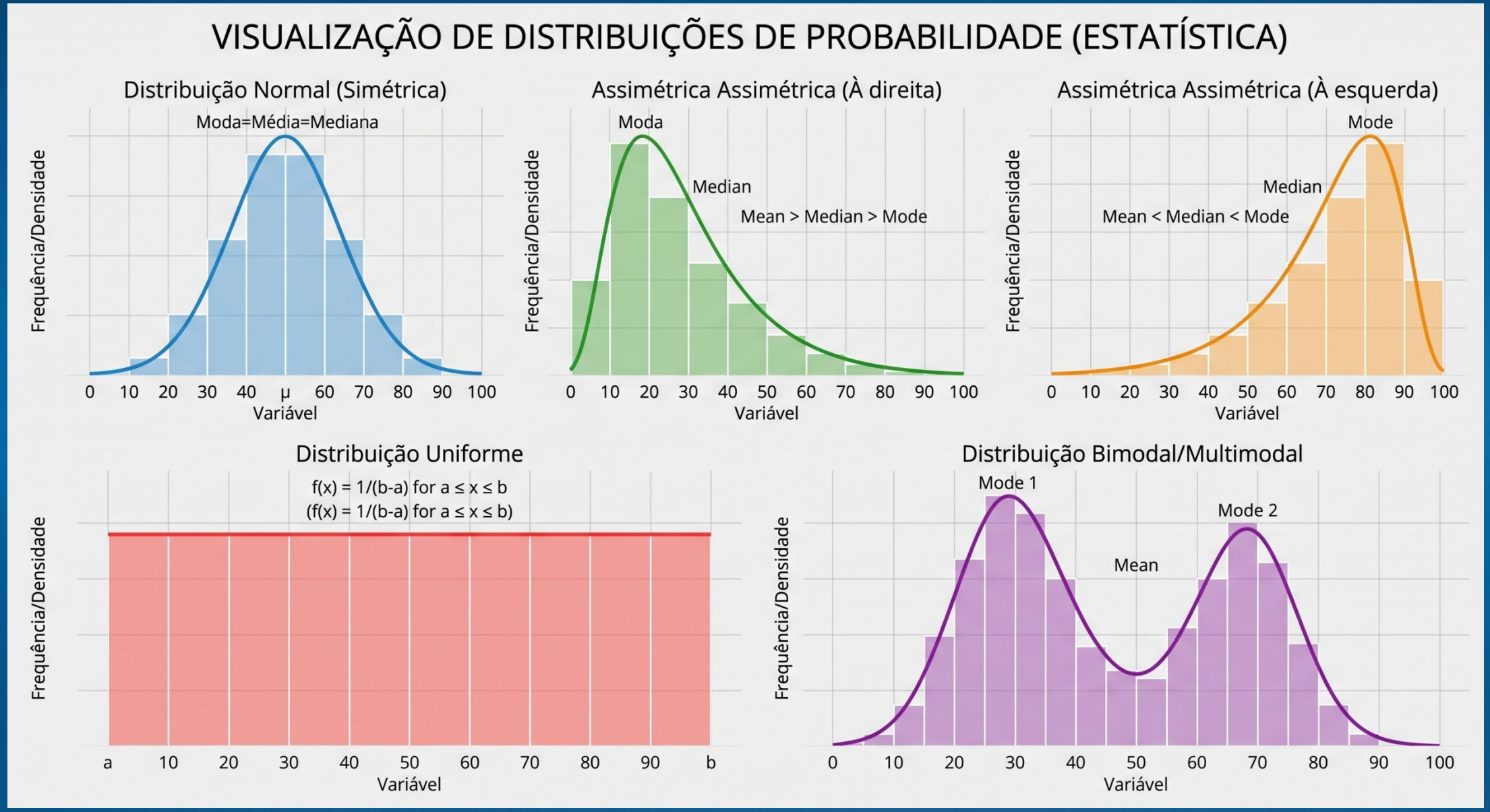
Imputar dados numéricos

Valores numéricos:

- **Mediana (.median())** ➔ Para variáveis numéricas com distribuição assimétrica ou presença de outliers (ex: renda, preços de imóveis), pois não sofre distorção por valores discrepantes.

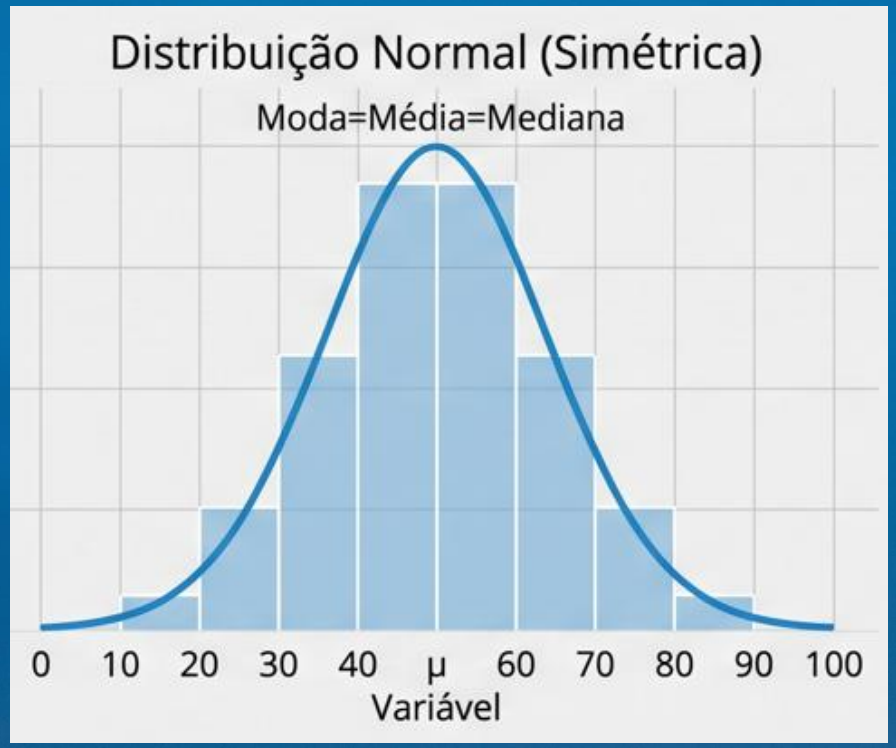


Distribuição



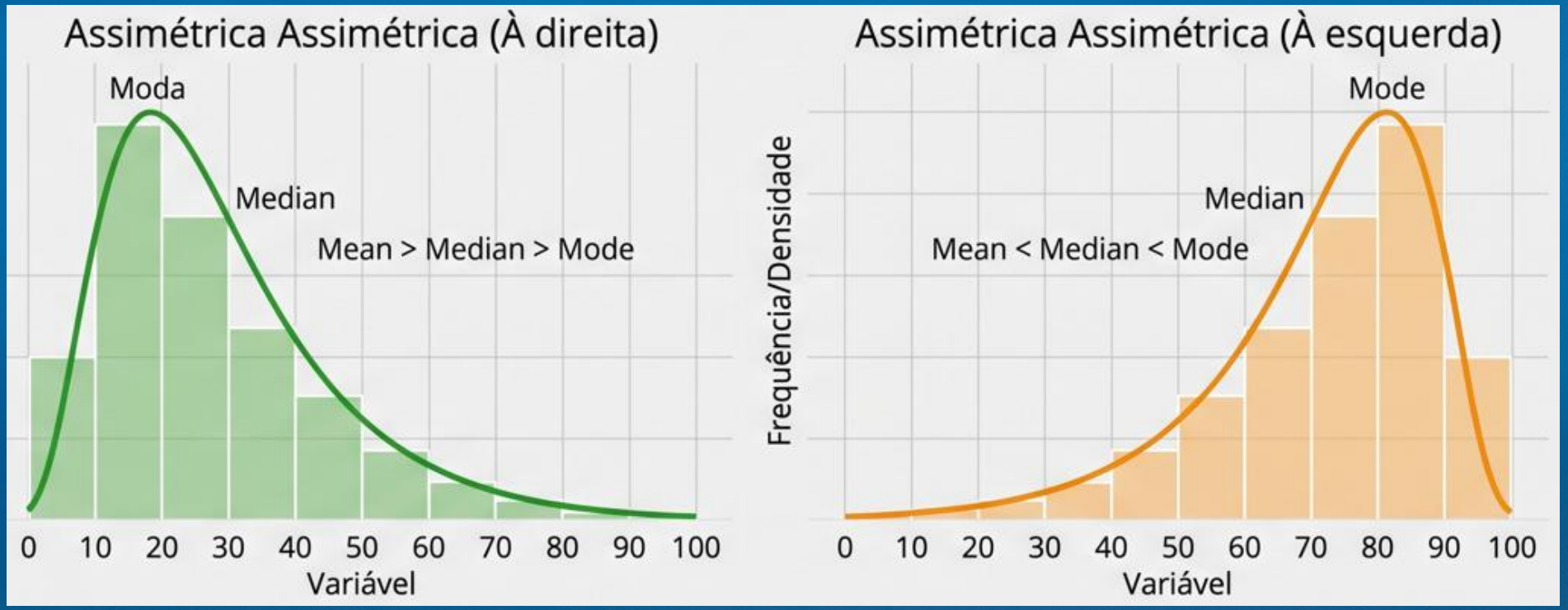
Distribuição Normal (Simétrica)

1. **Distribuição Normal (Simétrica):** Formato de "sino". A maioria dos valores está perto do centro (ex: altura, peso, pressão arterial).
    - **Decisão de Imputação:** Use a **Média**.



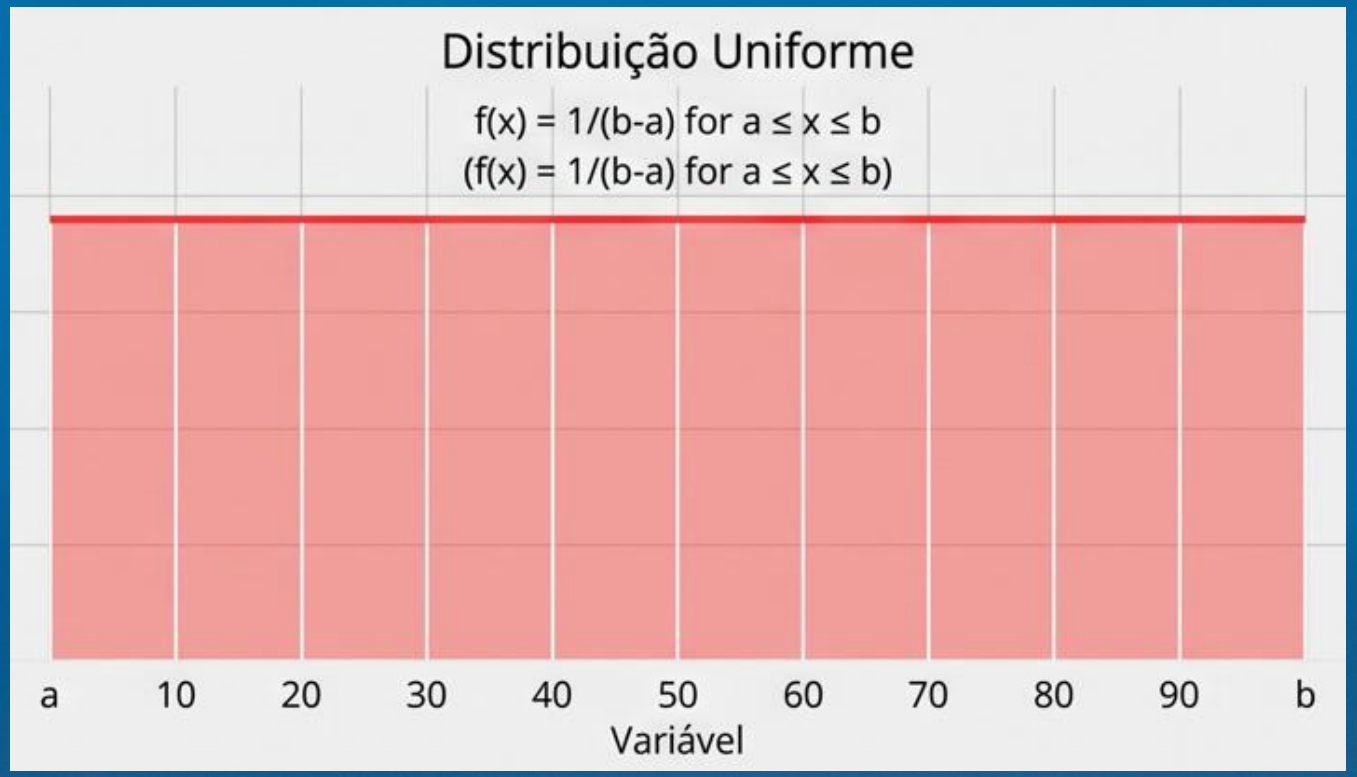
Distribuição Assimétrica

2. **Distribuição Assimétrica (À direita ou à esquerda):** Tem uma "cauda" puxada por valores extremos ou limites naturais (ex: renda, preço de imóveis, número de filhos).
   - **Decisão de Imputação:** Use a **Mediana**.



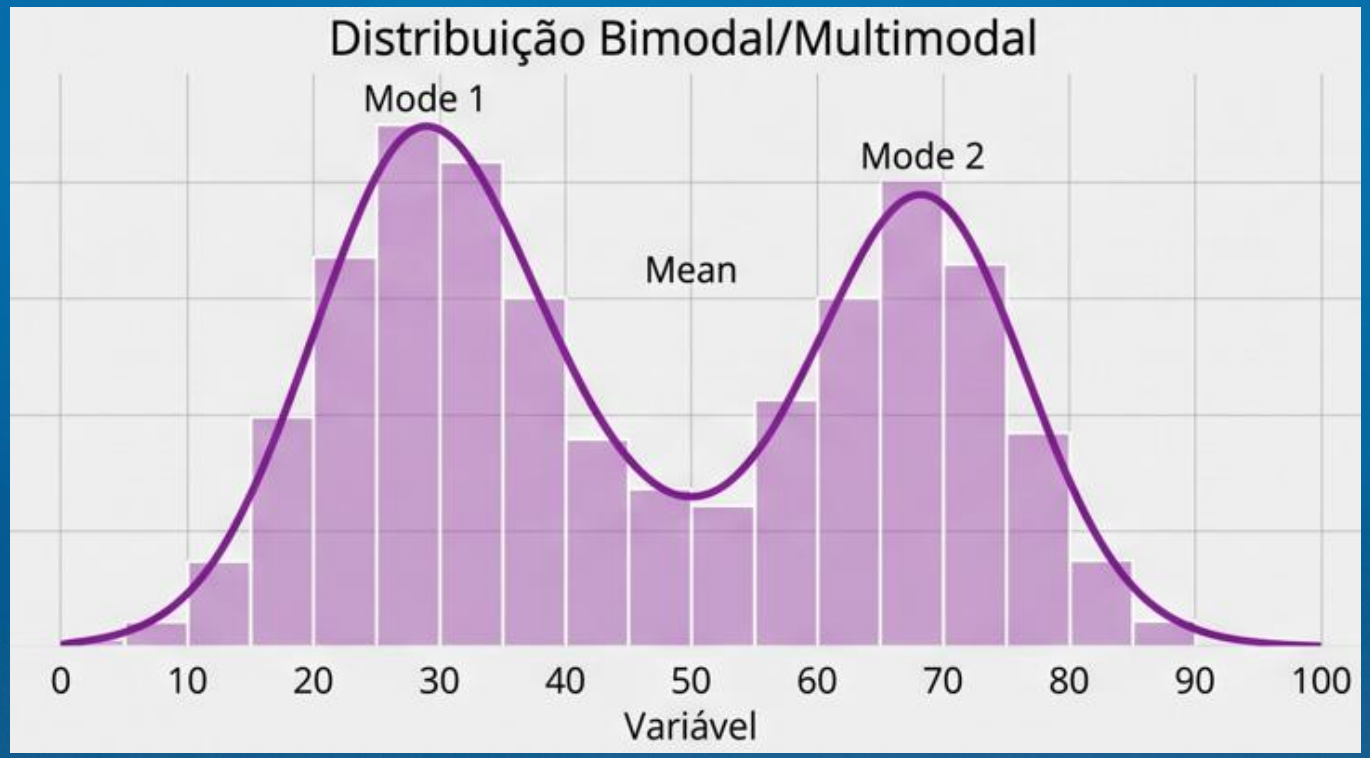
Distribuição Uniforme

3. **Distribuição Uniforme:** Todos os valores têm probabilidade parecida de ocorrer (ex: idade de clientes em um parque aquático bem variado).
   - **Decisão de Imputação:** Pode usar **Média** ou **Mediana**.



Distribuição Bimodal/Multimodal

4. **Distribuição Bimodal/Multimodal:** Possui dois ou mais "picos" de concentração (ex: altura de uma turma mista de adultos e crianças).
    - **Decisão de Imputação: Evite imputação simples**; prefira agrupar os dados (`groupby`) antes de imputar.



Código

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import math

#============================================================
#                    L E I T U R A
#============================================================
# Carregando o Iris Dataset baixado do Kaggle
df = pd.read_csv("Dados/07 - Inscritos.csv")

#============================================================
#                   INSPEÇÃO GERAL
#============================================================
# print("\nTamanho do Dataset (Linhas, Colunas):")
# print(df.shape)

# print("\n\nTipo das Variáveis\nDados:")
# print(df.dtypes)

# print("\nSeleção linhas:")
# print(df.head(10))

# print("\nResumo Estatístico (Média, Mín, Máx...):")
# print(df.describe())
```

Código

```
#============================================================
#              INSPEÇÃO DE VALORES AUSENTES
#============================================================
percentual_nulos_col = (df.isna().sum() / len(df)) * 100
#print(f"\n\nPercentual de nulos por coluna:\n {percentual_nulos_col}")

#============================================================
#               Identificação de Distribuição
#                   GRÁFICO HISTOGRAMA
#============================================================
altura = df['altura_m']
salario = df['salario_brl']

#print(f"\n\nAltura: {altura}\n Salário: {salario}")

# Configura a figura com 2 gráficos lado a lado
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

Código

```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Gráfico 1: ALTURA
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# # bins: Divide os dados em 15 colunas/intervalos de
# #       igual largura para mostrar a distribuição.
# axes[0].hist(altura,
#              color="skyblue",
#              edgecolor="black",
#              bins=15
#              )
# axes[0].set_title("Distribuição da Altura (Simétrica / Normal)")

# # Define início e fim do eixo X (ex: 1.50m a 2.00m)
# axes[0].set_xlim(1.67, 1.76)

# # Define início e fim do eixo Y (frequência)
# axes[0].set_ylim(0, 1.3)
```

Código

```
# #~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# # Gráfico 1: SALÁRIO
# #~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# # bins ==> Define a faixa de valores usada pelo bins.
# faixas_salario = [0, 5000, 10000, 20000, 40000, 60000, 80000, 100000]

# # Plota o histograma usando suas faixas
# axes[1].hist(salario,
#              color="salmon",
#              edgecolor="black",
#              bins=faixas_salario # Faixa de valores definidas anteriormente
#              )

# # Define início e fim do eixo X
# # Valores definidos para o bins
# axes[1].set_xticks(faixas_salario)

# # Define início e fim do eixo Y (frequência)
# axes[1].set_ylim(0, 4.5)

# # Opcional: rotaciona os números se eles ficarem muito próximos
# axes[1].tick_params(axis='x', rotation=45)

# # Ajusta automaticamente o espaçamento para que
# # títulos, eixos, rótulos e legendas não fiquem
# # sobrepostos nem cortados nas bordas da imagem.
# plt.tight_layout()

# plt.show()
```

Código

```
#============================================================
#              Identificação de Distribuição
#               COEFICIENTE DE ASSIMETRIA
#============================================================
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Coeficiente de assimetria: ALTURA
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# coef_assim_altura = df["altura_m"].skew()

# # Trunca o valorem uma casa decimal
# coef_assim_altura = math.floor(coef_assim_altura * 10) / 10
# print(f"\n\nAltura: Coeficiente de Assimetria => {coef_assim_altura}")

# if coef_assim_altura >= -0.5 and coef_assim_altura <= 0.5:
#     print(f"Altura: Decisão de Imputação => Usar a Média")
# else:
#     print(f"Altura: Decisão de Imputação => Usar a Mediana")
```

Código

```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Coeficiente de assimetria: SALÁRIO
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# coef_assim_salario = df["salario_brl"].skew()

# # Trunca o valorem uma casa decimal
# coef_assim_salario = math.floor(coef_assim_salario * 10) / 10
# print(f"\n\nSalario: Coeficiente de Assimetria => {coef_assim_salario}")

# if coef_assim_salario >= -0.5 and coef_assim_salario <= 0.5:
#     print(f"Salário: Decisão de Imputação => Usar a Média")
# else:
#     print(f"Salário: Decisão de Imputação => Usar a Mediana")
```

Código

```
#=============================================================
#           EFUTUAR IMPUTAÇÃO DADOS NUMÉRICOS
#=============================================================
print("\n\nDataFrane Antes:")
print(df)

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# ALTURA
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Obtém a MÉDIA
media_altura = df["altura_m"].mean()

# Preenche os nulos com a MÉDIA
df["altura_m"] = df["altura_m"].fillna(media_altura)

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# SALÁRIO
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Obtém a MEDIANA
mediana_salario = df["salario_brl"].median()

# Preenche os nulos com a MEDIANA
df["salario_brl"] = df["salario_brl"].fillna(mediana_salario)
```

Imputar dados categóricos

Valores categóricos (texto)

- **Moda (.mode()[0])** → Para variáveis categóricas (texto) ou discretas, substituindo pelo valor mais frequente da coluna.

Código

```
#=========================================================
#     EFUTUAR IMPUTAÇÃO DADOS CATEGÓRICOS - TEXTO
#=========================================================
# Retorna o valor mais frequente para UF
moda_uf = df["uf"].mode()[0]
print(f"\n\n UF:\n{moda_uf}")

# Preenche os nulos com a moda
df["uf"] = df["uf"].fillna(moda_uf)


print("\n\nDataFrane Depois:")
print(df)
```

Atividade Fixação

- Exercício 1: Diagnóstico e Remoção Crítica
- **Exercício 2: Escolha da Métrica Estatística**
- Exercício 3: Tratamento de Séries Temporais

Imputar dados sequencias e Temporais

Valores sequencias e Temporais

- Ordem cronológica evita inconsistências nos dados.
- Ideal para sensores IoT e finanças.
- Aplicável a dados climáticos e telemetria.

Imputar dados sequencias e Temporais

Métodos

- **Método ffill (forward fill)** → Propaga o último valor válido conhecido para as posições nulas seguintes.
- **Método bfill (backward fill)** → Busca o próximo valor válido futuro para preencher os nulos anteriores.
- **Interpolação linear com interpolate()** → Calcula a tendência matemática entre o ponto anterior e o posterior.

Código

```
import pandas as pd
import matplotlib.pyplot as plt



#=============================================================
#                    L E I T U R A
#=============================================================
# Carregando o Iris Dataset baixado do Kaggle
df = pd.read_csv("Dados/08 - Transmissor Temperatura.csv")

#=============================================================
#                   INSPEÇÃO GERAL
#=============================================================
# print("\nTamanho do Dataset (Linhas, Colunas):")
# print(df.shape)

# print("\n\nTipo das Variáveis\nDados:")
# print(df.dtypes)

# print("\nSeleção linhas:")
# print(df.head(10))

# print("\nResumo Estatístico (Média, Mín, Máx...):")
# print(df.describe())
```

Código

```
#============================================================
#          IMPUTAÇÃO SEQUENCIAL E TEMPORAL
#============================================================
print("\n\nDataFrane Antes:")
print(df)

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
#                   Forward Fill (ffill)
# Repete o último valor válido conhecido para cobrir
# os nulos seguintes
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# df_temp = df.copy()
# df_temp["temperatura_c"] = df_temp["temperatura_c"].ffill()

# print(f"\n\nForward Fill (ffill): \n{df_temp}")

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
#                Backward Fill (bfill)
# Pega o próximo valor válido futuro e preenche para trás
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# df_temp = df.copy()
# df_temp["temperatura_c"] = df_temp["temperatura_c"].bfill()

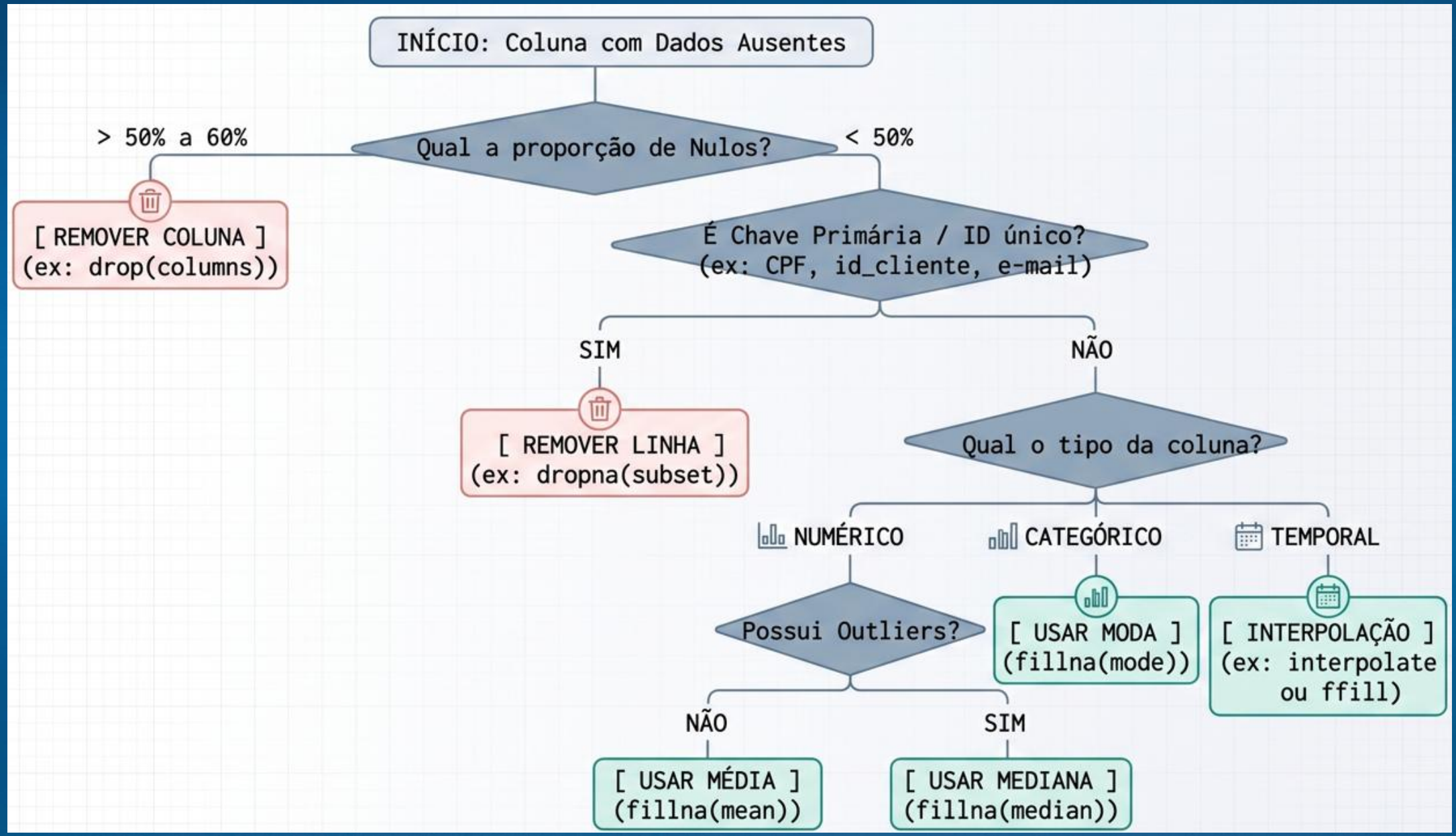
# print(f"\n\nBackward Fill (bfill): \n{df_temp}")
```

Código

```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# # 4. Testando a Interpolação Linear (interpolate)
# # Traça uma linha reta entre 21.0 e 24.0, resultando em 22.0 e 23.0
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
df_temp = df.copy()
df_temp["temperatura_c"] = df_temp["temperatura_c"].interpolate()

print(f"\n\nInterpolação Linear: \n{df_temp}")
```

Dados ausentes: Fluxo de decisão



Atividade Fixação

- Exercício 1: Diagnóstico e Remoção Crítica
- Exercício 2: Escolha da Métrica Estatística
- **Exercício 3: Tratamento de Séries Temporais**

F I M

"Nossa história, depositada nas mãos de Deus, pode ser reescrita a qualquer momento."

— PE. LUÍS ERLIN (9 MESES COM MARIA)

Prof. Dr. Ricardo Luis Balieiro
ricbalieiro@gmail.com